REACT

Using setTimeout in React components (including hooks)

Using timeouts in React isn't as straightforward as you may think. In this article, you'll learn how to avoid mistakes when using them in React.

Using the `setTimeout` function works the same in React as it does in plain JavaScript.

However, there are some caveats that you need to be aware of when using it in React that I want to get into in this tutorial.

Stick around for the last section, where we'll look at a better way of handling timeouts with React hooks.

Table of contents

- How to use setTimeout in React
- Using state in setTimeout
- Declarative timeouts with the useTimeout hook
- Conclusion

How to use setTimeout in React

The `setTimeout` function accepts two arguments: the first is the callback function that we want to execute, and the second specifies the timeout in milliseconds before the function will be called.

```javascript
setTimeout(() => console.log('Initial timeout!'), 1000);
```

In React, we use it the same way. Be careful, though, not to just place it anywhere in your function components, as this may run on every re-render.

If we want to execute the timeout **once** when the component mounts, we need to use `useEffect` for that:

```javascript
useEffect(() => {
  const timer = setTimeout(() => console.log('Initial timeout!'), 1000);
}, []);
```

The example can be found in this Codepen.

This already leads us to the next important part: **always clear the timeout**.

Always clear the timeout

If we don't clear our timeouts when the component unmounts, the code in the callback may execute even when the component isn't visible anymore.

It can also lead to **memory leaks** in your application. Since the timeout is still active after the component unmounts, the garbage collector won't collect the component.

> Read my article on JavaScript's memory management if you want to know how this works in detail.

Error messages like this one will pop up in the console:

```
5 ▸Can't call "this.setState" on an unmounted component. This is a no-op, but it debug.module.js:1
  indicates a memory leak in your application. To fix, cancel all subscriptions and asynchronous
  tasks in the componentWillUnmount method.

    in Image (at /Users/felix/version-control/felixgerschau.com/src/components/PostTileSmall/PostTi
  leSmall.tsx:29)
```

Can't call "this.setState" on an unmounted component.

To clear a timeout, we need to call `clearTimeout` with the returned value of `setTimeout`:

```javascript
const timer = setTimeout(() => console.log('Initial timeout!'), 1000);
clearTimeout(timer);
```

Function components

We can use the `useEffect` for running code when the component unmounts as well.

We can return a function in the callback that will run when the component unmounts. We'll use this function to clear the timeout we created on mount.

```javascript
useEffect(() => {
  const timer = setTimeout(() => console.log('Initial timeout!'), 1000);
  return () => clearTimeout(timer);
}, []);
```

But what if we want to create a timeout *outside* of `useEffect`?

Doing the following **doesn't** work because `timer` will be redefined on every re-render:

```javascript
let timer;
const sendMessage = (e) => {
  e.preventDefault();
  timer = setTimeout(() => alert('Hey ??'), 1000);
}

useEffect(() => {
  // "timer" will be undefined again after the next re-render
  return () => clearTimeout(timer);
}, []);
```

This doesn't work.

Using React's state for this also doesn't make sense because it would trigger a re-render. We need someplace where the variable will persist without using state.

References do exactly that.

We can assign the timer to `timerRef.current` and access it when the component unmounts:

```javascript
const timerRef = useRef(null);
const sendMessage = (e) => {
  e.preventDefault();
  timerRef.current = setTimeout(() => alert('Hey ??'), 1000);
}

useEffect(() => {
  // Clear the interval when the component unmounts
  return () => clearTimeout(timerRef.current);
}, []);
```

We can be sure that the timeout function will only execute if the component is mounted.

> You can try out the examples in this Codepen. Try to click "Hide App component" immediately **after** "Send message". If it weren't for calling `clearTimeout`, you would still see the alert.

Class components

Persisting state between re-renders is much easier in class components. Here, we can create a new class property for the timer.

Then we clear the timeout in the lifecycle function `componentWillUnmount`:

```
class App extends Component {
  timer;

  sendMessage = (e) => {
    e.preventDefault();
    this.timer = setTimeout(() => alert('Hey ??'), 1000);
  }

  componentWillUnmount() {
    clearTimeout(this.timer);
  }

  render() {
    return (
      <button onClick={this.sendMessage}>
        Send message
      </button>
    );
```

```
  }
}
```

javascript

Using state in setTimeout

Using state variables in the `setTimeout` callback can be a bit counterintuitive.

Let's take the following code as an example. You can type a message in the input field; clicking "Send message" displays an alert after two seconds.

```
const App = () => {
  const [message, setMessage] = useState('');
  const handleChange = (e) => {
    e.preventDefault();
    setMessage(e.target.value);
  };

  const sendMessage = (e) => {
    e.preventDefault();
    setTimeout(() => {
      alert(message);
    }, 2000);
  };

  return (
    <>
      <input onChange={handleChange} value={message} />
      <button onClick={sendMessage}>
        Send message
      </button>
    </>
```

```
  );
};
```
javascript

If you change the input value immediately **after** you click "Send message", *will the timeout display the updated value, or will it take the last value that was available when you clicked the button?*

> You can try it out here.

As stated in this GitHub issue, `setTimeout` will use the value that it was **initially** called with. In our example, this also makes sense as you wouldn't want to send another message other than the one that was displayed when you clicked the button.

If we want to get the **most recent value**, we need to make use of references.

We first create a new reference with the `useRef` hook and then use `useEffect` to listen to changes of the `message` variable.

Every time the variable changes, we'll assign the reference to the value of the state.

We'll then use the reference instead of the state variable in the timeout to get the latest version.

```
const App = () => {
  const messageRef = useRef('');
  const [message, setMessage] = useState('');

  useEffect(() => {
```

```javascript
    messageRef.current = message;
  }, [message]);

  const handleChange = (e) => {
    e.preventDefault();
    setMessage(e.target.value);
  };

  const sendMessage = (e) => {
    e.preventDefault();
    setTimeout(() => {
      // Most recent value
      alert(messageRef.current);
    }, 2000);
  };

  return (
    <>
      <input onChange={handleChange} value={message} />
      <button onClick={sendMessage}>
        Send message
      </button>
    </>
  )
}
```

javascript

Declarative timeouts with the useTimeout hook

Similar to the useInterval hook, creating a custom `useTimeout` hook makes working with timeouts in React easier.

Abstracting away the creation and clearing of timeouts makes using them more manageable and safer since we don't have to remember clearing them

every time.

```javascript
useTimeout(() => {
  // Do something
}, 5000);
```

Just like `setTimeout`, the hook accepts a callback and a number.

Setting the number to `null` will disable the timeout, and it also automatically cancels it when the component unmounts.

The code of the hook looks like the following.

```javascript
import { useEffect, useRef } from 'react'

function useTimeout(callback, delay) {
  const savedCallback = useRef(callback)

  // Remember the latest callback if it changes.
  useEffect(() => {
    savedCallback.current = callback
  }, [callback])

  // Set up the timeout.
  useEffect(() => {
    // Don't schedule if no delay is specified.
    if (delay === null) {
      return
    }

    const id = setTimeout(() => savedCallback.current(), delay)

    return () => clearTimeout(id)
  }, [delay])
}
```

Props to the useHooks website for providing the code for this hook.

Note how the hook clears the timeout in the returned callback function of the `useEffect` hook. This will make sure that it always clears when `delay` changes or when the component unmounts.

On the Overreacted blog, Dan Abramov explains in detail why this approach integrates so well with how we write React code.

Conclusion

As you can see in this article, you need to be aware of several things when using timeouts in React.

Still, you can get around most of the issues by using the `useInterval` hook instead of using the JavaScript API every time as it handles clearing the timeout for you.

If this article helped you, sign up for my newsletter to get notified when I publish new articles about React and JavaScript.

Examples used in this tutorial:

- setTimeout example (Codepen)
- Using setTimeout with state (Codepen)
- The useTimeout hook (usehooks-typescript.com)

Other articles you might like:

- Separation of concerns with React hooks

Get notified about new tutorials

Join over 1,000 developers who receive React and JavaScript tutorials via email.

Enter your email

Notify me

No spam. Unsubscribe at any time.

2 Comments - *powered by utteranc.es*

tsaomaxwell commented on 10. Sept. 2022

This is awesome, I was working on time specific handling for duplicate events, and was getting stuck on state and timeout. Not only did this article fix the problem for me, it also made my solution more elegant and taught me a lot about how react works in a very understandable way. Thanks!

fgerschau commented on 10. Sept. 2022 Owner

@tsaomaxwell thanks for the nice words. I'm glad that you found it useful!

Write Preview

Sign in to comment

Styling with Markdown is supported

Sign in with GitHub

© 2022 Felix Gerschau. All rights reserved.